Solved By :- Prem .

Course Code : MCSL-209

Course Title : Data Structures and Algorithms Lab

Assignment Number : BCA_NEW(III)L-209/Assignment/2025-26

Question 1: Write an algorithm and program in 'C' language to merge two sorted linked lists. The resultant linked list should be sorted.

ANS:-

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}

// Function to print a linked list
void printList(struct Node* head) {
   while (head != NULL) {
      printf("%d -> ", head->data);
      head = head->next;
   }
```

```
    printf("NULL\n");
}

// Function to merge two sorted linked lists
struct Node* mergeLists(struct Node* head1, struct Node* head2) {
    // Dummy node
    struct Node dummy;
    struct Node* tail = &dummy;
    dummy.next = NULL;

    // Traverse both lists
    while (head1 != NULL && head2 != NULL) {
        if (head1->data <= head2->data) {
            tail->next = head1;
            head1 = head1->next;
        } else {
            tail->next = head2;
            head2 = head2->next;
        }
        tail = tail->next;
    }

    // Attach remaining nodes
    if (head1 != NULL) {
        tail->next = head1;
    } else {
        tail->next = head2;
    }

    return dummy.next;
}
```

```
// Main function
int main() {
    // First sorted linked list: 1 -> 3 -> 5
    struct Node* head1 = createNode(1);
    head1->next = createNode(3);
    head1->next->next = createNode(5);

    // Second sorted linked list: 2 -> 4 -> 6
    struct Node* head2 = createNode(2);
    head2->next = createNode(4);
    head2->next->next = createNode(6);

    printf("First Linked List: ");
    printList(head1);

    printf("Second Linked List: ");
    printList(head2);

    // Merge both lists
    struct Node* mergedHead = mergeLists(head1, head2);

    printf("Merged Sorted Linked List: ");
    printList(mergedHead);

    return 0;
}
```

OUTPUT

```
Output
First Linked List: 1 -> 3 -> 5 -> NULL
Second Linked List: 2 -> 4 -> 6 -> NULL
Merged Sorted Linked List: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL
```

Question 2 : Write an algorithm and a program in 'C' language to insert and delete edges in an adjacency list representation of an undirected graph. Make assumptions, if necessary.

Ans:-

We need to:

1. Represent an **undirected graph** using adjacency lists.
2. Write an algorithm and C program to **insert** and **delete** edges.

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for adjacency list
struct Node {
    int vertex;
    struct Node* next;
};

// Graph structure
struct Graph {
    int numVertices;
    struct Node** adjLists; // Array of adjacency lists
};

// Function to create a new node
```

```
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

// Function to create a graph
struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = (struct Node**)malloc(vertices * sizeof(struct Node*));

    for (int i = 0; i < vertices; i++)
        graph->adjLists[i] = NULL;

    return graph;
}

// Function to add an edge (undirected)
void addEdge(struct Graph* graph, int src, int dest) {
    // Add edge from src to dest
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    // Add edge from dest to src (since undirected)
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}
```

```
// Function to delete an edge
void deleteEdge(struct Graph* graph, int src, int dest) {
    struct Node* temp = graph->adjLists[src];
    struct Node* prev = NULL;

    // Delete dest from src's list
    while (temp != NULL && temp->vertex != dest) {
        prev = temp;
        temp = temp->next;
    }
    if (temp != NULL) {
        if (prev == NULL) // First node
            graph->adjLists[src] = temp->next;
        else
            prev->next = temp->next;
        free(temp);
    }

    // Delete src from dest's list
    temp = graph->adjLists[dest];
    prev = NULL;
    while (temp != NULL && temp->vertex != src) {
        prev = temp;
        temp = temp->next;
    }
    if (temp != NULL) {
        if (prev == NULL)
            graph->adjLists[dest] = temp->next;
        else
            prev->next = temp->next;
```

```
        free(temp);
    }
}

// Function to print the graph
void printGraph(struct Graph* graph) {
    for (int v = 0; v < graph->numVertices; v++) {
        struct Node* temp = graph->adjLists[v];
        printf("\nAdjacency list of vertex %d: ", v);
        while (temp) {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }
        printf("NULL");
    }
    printf("\n");
}

// Main function
int main() {
    int V = 5; // Assume 5 vertices (0,1,2,3,4)
    struct Graph* graph = createGraph(V);

    // Insert edges
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 4);
    addEdge(graph, 1, 2);
    addEdge(graph, 1, 3);
    addEdge(graph, 1, 4);
    addEdge(graph, 2, 3);
    addEdge(graph, 3, 4);
```

```
    printf("Graph after insertion of edges:");
    printGraph(graph);

    // Delete an edge
    printf("\nDeleting edge (1,4)\n");
    deleteEdge(graph, 1, 4);

    printf("Graph after deletion of edge (1,4):");
    printGraph(graph);

    return 0;
}
```

Output Before Deletion:

```
Adjacency list of vertex 0: 4 -> 1 -> NULL
Adjacency list of vertex 1: 4 -> 3 -> 2 -> 0 -> NULL
Adjacency list of vertex 2: 3 -> 1 -> NULL
Adjacency list of vertex 3: 4 -> 2 -> 1 -> NULL
Adjacency list of vertex 4: 3 -> 1 -> 0 -> NULL
```

After Deleting Edge (1,4):

rust

```
Adjacency list of vertex 0: 4 -> 1 -> NULL
Adjacency list of vertex 1: 3 -> 2 -> 0 -> NULL
Adjacency list of vertex 2: 3 -> 1 -> NULL
Adjacency list of vertex 3: 4 -> 2 -> 1 -> NULL
Adjacency list of vertex 4: 3 -> 0 -> NULL
```